

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний університет водного господарства та природокористування
Навчально-науковий інститут кібернетики, інформаційних технологій та
інженерії
Кафедра комп'ютерних наук та прикладної математики

КУРСОВА РОБОТА

з дисципліни «Програмування»

на тему:
Taskly To Do List App

Студент групи ПЗ-21
Зубар Назарій

Керівник: к.т.н., доц. Жуковський В.В.

Зміст

Вступ	2
1 Аналіз предметної області та теоретичні основи	3
1.1 Огляд технологій та підходів	3
1.2 Опис предметної області	3
1.3 Аналіз аналогів	4
2 Архітектура системи	5
2.1 Огляд Clean Architecture	5
2.2 Діаграма архітектури	5
2.3 Залежності проекту	6
3 Документація API	7
3.1 Опис REST API	7
3.1.1 Реєстрація користувача	7
3.1.2 Аутентифікація користувача	9
3.1.3 Створення нового завдання	10
3.2 Інші ендпоінти	11
4 Діаграми	12
4.1 Секвенційна діаграма	12
4.2 DFD діаграма	13
5 Інтерфейс користувача	14
5.1 Огляд UI додатку	14
5.1.1 Головна сторінка	14
5.1.2 Сторінка реєстрації	15
5.1.3 Інтерфейс дошки з завданнями	16
5.1.4 Інтерфейс створення завдання	17
6 Тестування та оцінка системи	18
6.1 Методологія тестування	18
6.2 Результати тестування	18
6.3 Продуктивність та масштабованість	18
7 Висновки	19
Список використаних джерел	20

Вступ

У сучасному світі ефективне управління часом і завданнями є критично важливим для підвищення продуктивності як у повсякденному житті, так і в професійній діяльності. Курсовий проєкт «Taskly To Do List App» присвячений розробці системи керування завданнями, яка дає змогу користувачам створювати, редагувати та контролювати виконання завдань.

У роботі розглядаються архітектура застосунку, розробка REST API, використання технології SignalR для реального часу, інтеграція інтерфейсу з бекендом, а також здійснюється оцінка тестування та продуктивності системи.

Метою даної роботи є створення повнофункціонального застосунку, який реалізує принципи Clean Architecture для побудови масштабованих, зручних у підтримці та тестуванні систем. Основні завдання включають аналіз предметної області, розробку архітектури, створення API-документації, реалізацію користувацького інтерфейсу та проведення тестування.

РОЗДІЛ 1. Аналіз предметної області та теоретичні основи

1.1 Огляд технологій та підходів

У сучасній розробці веб-додатків широкого поширення набули принципи Clean Architecture, які забезпечують чітке розділення відповідальностей між компонентами системи. Така архітектура сприяє масштабованості, тестованості та зручності підтримки застосунків. Основні шари, що визначають архітектурну модель, включають:

- **Доменний шар** – містить бізнес-логіку та основні сутності предметної області;
- **Шар застосування** – реалізує сервіси, обробку запитів і координацію між доменним шаром та зовнішніми інтерфейсами;
- **Інтерфейсний шар** – відповідає за взаємодію з користувачем через API, веб-інтерфейс або інші клієнтські застосунки;
- **Інфраструктурний шар** – забезпечує доступ до баз даних, зовнішніх сервісів, файлової системи та мережевих ресурсів.

У рамках реалізації проєкту застосовуються такі технології:

- **Серверна частина (Backend):** мова програмування C# у середовищі .NET, використання механізмів залежностей (Dependency Injection), побудова REST API та підтримка взаємодії в реальному часі за допомогою SignalR;
- **Клієнтська частина (Frontend):** мова TypeScript, бібліотека React, а також сучасні засоби взаємодії з API (зокрема, бібліотека Axios);
- **Бази даних:** SQL Server або альтернативні NoSQL-рішення залежно від вимог до гнучкості й масштабованості;
- **Забезпечення безпеки:** реалізація аутентифікації та авторизації за допомогою JWT (JSON Web Token) і протоколу OAuth.

1.2 Опис предметної області

Taskly To Do List App — це потужний веб-інструмент для організації та управління завданнями, який підходить як для особистого використання, так і для командної співпраці. Основна мета системи — забезпечити простий та інтуїтивно зрозумілий інтерфейс, який дозволяє створювати, редагувати й відстежувати завдання в реальному часі.

Системи типу to-do list широко застосовуються у сфері особистої продуктивності, проєктного менеджменту, навчання, розробки програмного забезпечення тощо. Вони дають

змогу структурувати робочі процеси, підвищити відповідальність за виконання задач і зменшити ризики пропущених дедлайнів.

Особливу актуальність такі застосунки мають в умовах гнучкого графіка, віддаленої роботи та швидкозмінного середовища, де важливо своєчасно реагувати на нові завдання. У цьому контексті Taskly вирізняється підтримкою роботи в реальному часі (SignalR), що дозволяє команді бачити зміни в задачах миттєво, без перезавантаження сторінки.

Таким чином, предметна область проекту охоплює цифрові системи управління завданнями, які інтегрують сучасні підходи до продуктивності, співпраці та взаємодії з даними.

1.3 Аналіз аналогів

Серед систем типу to-do list існує значна кількість застосунків, одними з найпопулярніших є Trello, Microsoft To Do тощо. У таблиці 1.1 наведено порівняння основних характеристик деяких із них.

Табл. 1.1: Порівняльна характеристика популярних систем управління завданнями

Характеристика	Trello	Microsoft To Do	Taskly
Інтерфейс	Канбан-дошка	Списки завдань	Канбан-дошка
Підтримка командної роботи	Так	Обмежена	Так
Синхронізація в реальному часі	Так	Так	Так
Наявність гейміфікації	Можлива через розширення	Ні	Так
Безкоштовна версія	Так	Так	Так
Наявність системи винагород	Ні	Ні	Так
Використання ШІ	Ні	Ні	Так

РОЗДІЛ 2. Архітектура системи

2.1 Огляд Clean Architecture

Clean Architecture - це шаблон проектування програмного забезпечення, який підкреслює розділення завдань, роблячи застосунок більш модульним, тестованим та зручним у підтримці. Він організовує застосунок на окремі шари з чіткими межами, що забезпечує більшу гнучкість та масштабованість. Ядро застосунку, що містить бізнес-логіку, не залежить від фреймворків, баз даних або інтерфейсу користувача.

- **Доменний шар (Domain Layer):** містить основну бізнес-логіку, сутності та варіанти використання. Він не залежить від будь-якого конкретного фреймворку чи технології.
- **Шар застосування (Application layer):** взаємодіє з рівнем домену. Він визначає бізнес-логіку, специфічну для застосунку.
- **Інтерфейсний шар (Presentation Layer):** обробляє взаємодію інтерфейсу користувача та зв'язок з рівнем додатку.
- **Інфраструктурний шар (Infrastructure layer):** реалізує інтерфейси, визначені на рівні домену, обробляючи зовнішні залежності, такі як бази даних та API.

2.2 Діаграма архітектури

Нижче наводиться схема архітектури системи Taskly:

РОЗДІЛ 3. Документація API

3.1 Опис REST API

Додаток Taskly реалізує набір REST API для управління завданнями. Далі наведено приклади основних ендпоінтів.

3.1.1 Реєстрація користувача

Реєстрація проходить в 3 етапи, а саме

- **Надсилання верифікаційного коду:** користувач вказує ел. адресу, на яку хоче зареєструвати акаунт, на неї надсилається код підтвердження.
- **Верифікація електронної адреси:** користувач вводить код підтвердження, який прийшов йому на пошту.
- **Кінцева реєстрація:** після успішної верифікації, користувач надає решту необхідної інформації та завершує реєстрацію.

Надсилання верифікаційного коду

Метод: POST

URL: /api/authentication/send-verification-code

Запит:

```
1 POST /api/authentication/send-verification-code HTTP/1.1
2 Host: taskly-backend-0int.onrender.com
3 Content-Type: application/json
4
5 {
6   "email": "ten3ee.n@gmail.com"
7 }
```

Лістинг 3.1: Запит на надсилання коду підтвердження

Відповідь:

```
1 HTTP/1.1 200 Success
2 Content-Type: application/json
3
4 {
5   "email": "ten3ee.n@gmail.com"
6 }
```

Лістинг 3.2: Відповідь на надсилання коду підтвердження

Верифікація електронної адреси

Метод: POST

URL: /api/authentication/verificate-email

Запит:

```
1 POST /api/authentication/verificate-email HTTP/1.1
2 Host: taskly-backend-0int.onrender.com
3 Content-Type: application/json
4
5 {
6   "email": "ten3ee.n@gmail.com",
7   "code": "3730516"
8 }
```

Лістинг 3.3: Запит на верифікацію електронної адреси

Відповідь:

```
1 HTTP/1.1 200 Success
2 Content-Type: application/json
3
4 {
5   "email": "ten3ee.n@gmail.com"
6 }
```

Лістинг 3.4: Відповідь на верифікацію електронної адреси

Кінцева реєстрація

Метод: POST

URL: /api/authentication/register

Запит:

```
1 POST /api/authentication/register HTTP/1.1
2 Host: taskly-backend-0int.onrender.com
3 Content-Type: application/json
4
5 {
6   "email": "ten3ee.n@gmail.com",
7   "password": "NazariiZubar_345",
8   "confirmPassword": "NazariiZubar_345",
9   "avatarId": "FC2F7E84-A633-4DB0-929A-BC2DFDF57FD8"
10 }
```

Лістинг 3.5: Запит на реєстрацію акаунту

Відповідь:

```
1 HTTP/1.1 200 Success
2 Content-Type: application/json
3
4 {
5 }
```

Лістинг 3.6: Відповідь на реєстрацію акаунту

3.1.2 Аутентифікація користувача

Метод: POST

URL: /api/authentication/login

Запит:

```
1 POST /api/authentication/login HTTP/1.1
2 Host: taskly-backend-0int.onrender.com
3 Content-Type: application/json
4
5 {
6   "email": "nazarzubar345@gmail.com",
7   "password": "NazariyZubar_345",
8   "rememberMe": false
9 }
```

Лістинг 3.7: Запит на аутентифікацію користувача

Відповідь:

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3
4 {
5   "$id": "1",
6   "id": "e09cac90-2a46-40a9-8603-88491e306282",
7   "email": "nazarzubar345@gmail.com",
8   "avatarName": "flamingo",
9   "token": "
10     eyJhbGciOiJodHRwOi8vd3d3LnczLm9yZy8yMDAxLzA0L3htbGRzaWctbW9yZSNoWFj
11     LXNoYTI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImUwOWNhYzkwLTJhNDYtNDBhOS04NjA
12     zLTg4NDkxZTMwNjI4MiIsImVtYWlsIjoibmF6YXJ6dWJhcjM0NUBnbWFpbC5jb20iLCJ
13     odHRwOi8vc2NoZW1hcy5taWVyb3NvZnQuY29tL3dzLzIwMDgvMDYvaWRlbnRpdHkv
14     Y2xhaW1zL3JvbGUuOiJvc2VyIiwibmJmIjozNzQ5MDMyOTIyLCJleHAiOiE3NDkxMTkz
15     MjJ9.c_YIF_aAWKMQxIC_czWCJlKR1aSfevBtcGO0F5ikn4s",
16   "role": "User"
17 }
```

Лістинг 3.8: Відповідь на аутентифікацію користувача

3.1.3 Створення нового завдання

Метод: POST

URL: /api/Cards/create-card

Запит:

```
1 POST /api/Cards/create-card HTTP/1.1
2 Host: taskly-backend-0int.onrender.com
3 Content-Type: application/json
4 Authorization: Bearer
    eyJhbGciOiJodHRwOi8vd3d3LnczLm9yZy8yMDAxLzA0L3htbGRzaWctbW9yZSNoYWJj
5    LXRhYXNjaWkiOiJodHRwOi8vd3d3LnczLm9yZy8yMDAxLzA0L3htbGRzaWctbW9yZSNo
6    zLTg4NDkxZTMwNjI4MiIsImVtYWlsIjoibmF6YXJ6dWJhcjM0NUBnbWFpbC5jb20iLCJ
7    odHRwOi8vc2NoZW1hcy5taWVyb3NvZnQuY29tL3dzLzIwMDgvMDYvaWRlbnRpdHkvY2x
8    haW1zL3JvbGU0IjVc2VyIiwibmJmIjozNzQ5MDMyOTIyLCJleHAiOiJlE3NDkxMTkzMjJ
9    9.c_YIF_aAWKMqXIC_czWCJlKR1aSfevBtcGO0F5ikn4s
10
11 {
12   "cardListId": "61AC9C9F-98FA-4982-FAA3-08DDA1BDE37D",
13   "task": "My task",
14   "deadline": "2025-07-04T11:00:14.766Z",
15   "userId": "E09CAC90-2A46-40A9-8603-88491E306282"
16 }
```

Лістинг 3.9: Запит на створення завдання

Відповідь:

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3
4 {
5   "cardId": "e4971f23-879b-4730-adff-dc416e092f7d"
6 }
```

Лістинг 3.10: Відповідь на створення завдання

Після успішного створення картки, її дані надсилаються іншим користувачам, що перебувають на відповідній дошці, за допомогою технології SignalR. Це дозволяє здійснювати динамічне оновлення інтерфейсу в реальному часі без необхідності ручного перезавантаження сторінки.

3.2 Інші ендпоінти

Окрім вищенаведених, додаток також підтримує операції редагування, видалення та отримання списку завдань тощо. Ці ендпоінти реалізовані аналогічно до описаних раніше: використовують методи GET, PUT та DELETE, відповідно до REST-принципів. Їхній детальний опис не наводиться у цій роботі, оскільки основна увага зосереджена на ключовому функціоналі — реєстрації, аутентифікації та створенні завдань.

РОЗДІЛ 4. Діаграми

4.1 Секвенційна діаграма

Нижче наведено приклад секвенційної діаграми, що ілюструє процес створення завдання:

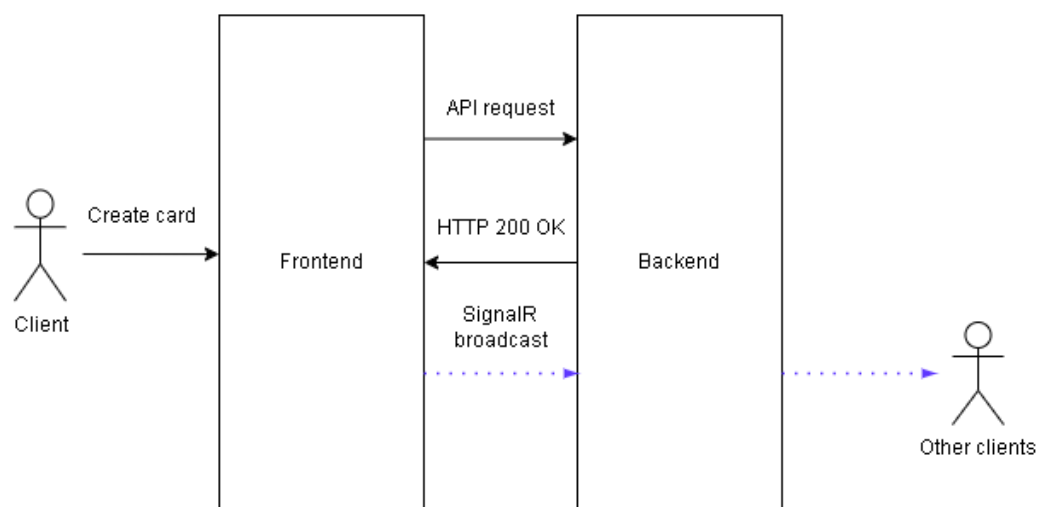


Рис. 4.1: Секвенційна діаграма створення завдання

4.2 DFD діаграма

DFD діаграма демонструє потік даних між основними компонентами системи:

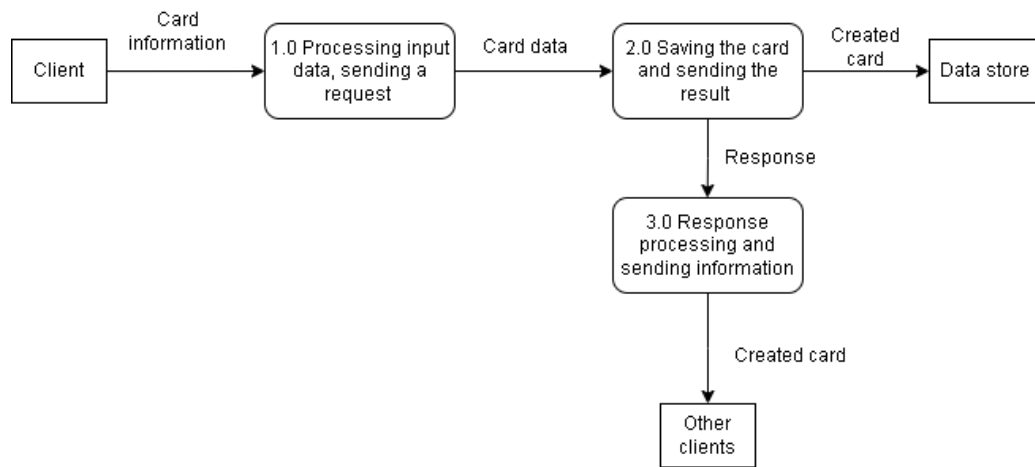


Рис. 4.2: Потік даних між компонентами

РОЗДІЛ 5. Інтерфейс користувача

5.1 Огляд UI додатку

Додаток має сучасний, інтуїтивно зрозумілий інтерфейс, що дозволяє:

- Легко створювати, редагувати та видаляти завдання.
- Реєструватися та входити до системи.
- Отримувати підтвердження виконаних операцій.

5.1.1 Головна сторінка

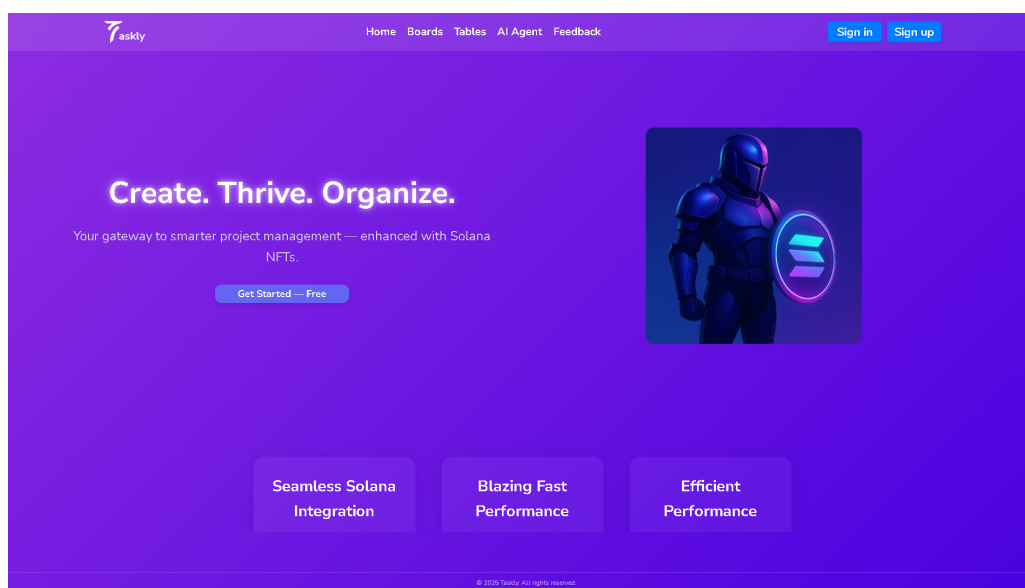


Рис. 5.1: Головна сторінка Taskly

5.1.2 Сторінка реєстрації

- Верифікація пошти:

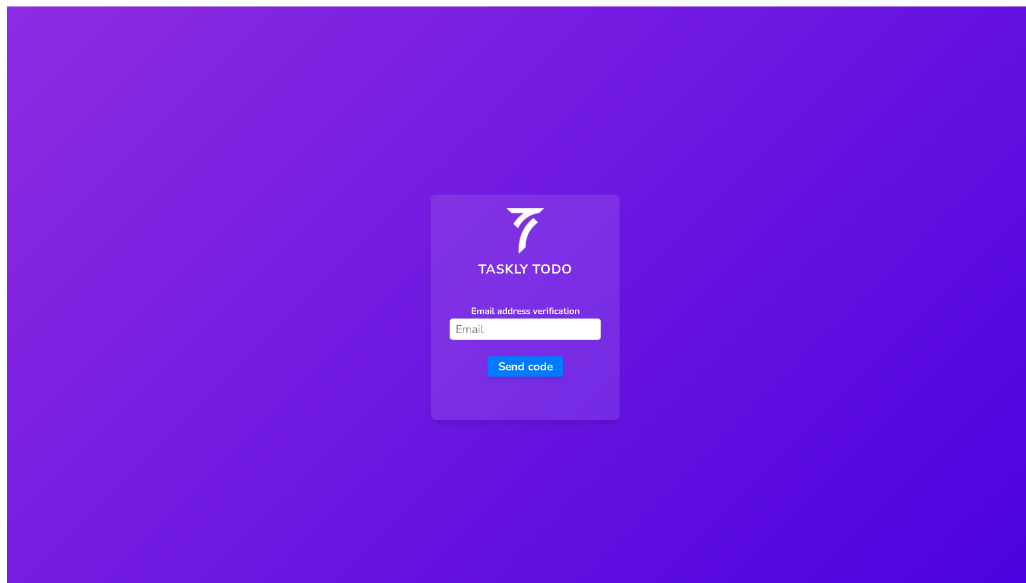
A screenshot of a web form for email verification. The form is centered on a solid blue background. It features a white square box containing the TASKLY TODO logo (a stylized '7' icon) and the text 'TASKLY TODO'. Below the logo, the text 'Email address verification' is displayed. Underneath, there is a white input field with the placeholder text 'Email'. At the bottom of the box is a blue button with the text 'Send code' in white.

Рис. 5.2: Форма ввод електронної пошти

- Підтвердження верифікаційного коду:

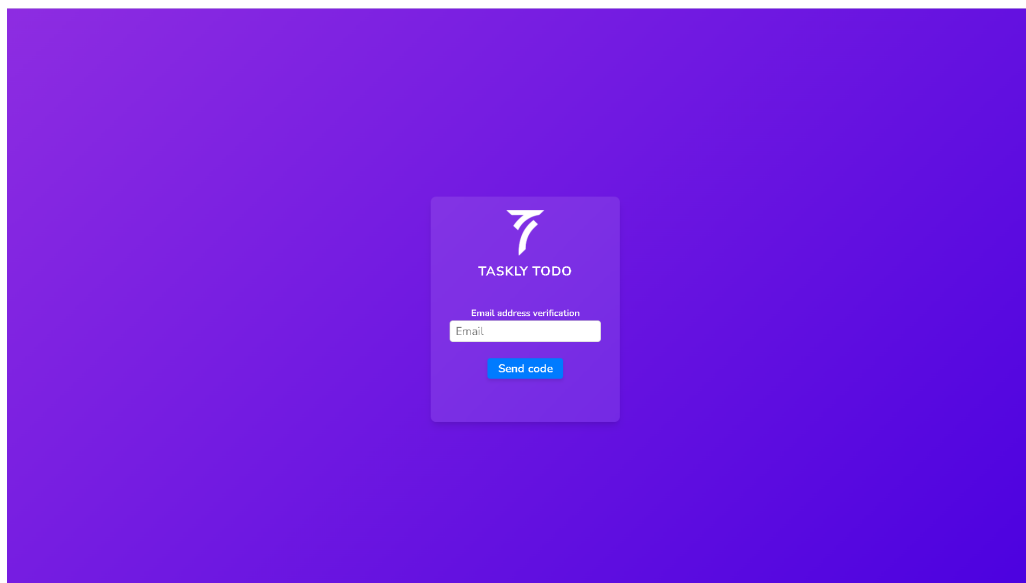
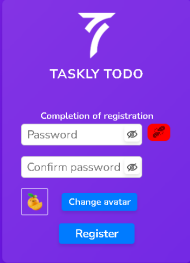
A screenshot of a web form for confirming a verification code. The form is centered on a solid blue background. It features a white square box containing the TASKLY TODO logo (a stylized '7' icon) and the text 'TASKLY TODO'. Below the logo, the text 'Email address verification' is displayed. Underneath, there is a white input field with the placeholder text 'Email'. At the bottom of the box is a blue button with the text 'Send code' in white.

Рис. 5.3: Форма вводу верифікаційного коду

- Завершення реєстрації:



The image shows a registration completion form for 'TASKLY TODO' on a dark blue background. The form is centered and contains the following elements: a logo at the top, the text 'Completion of registration', two input fields for 'Password' and 'Confirm password' (the 'Password' field has a red error icon), a 'Change avatar' button with a person icon, and a 'Register' button.

Рис. 5.4: Форма вводу решти необхідних даних

5.1.3 Інтерфейс дошки з завданнями

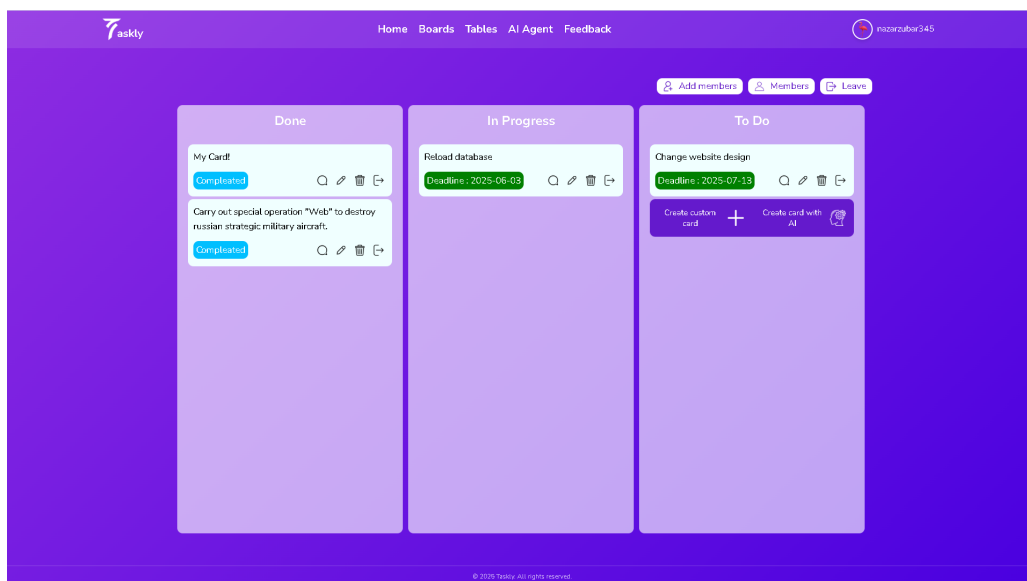
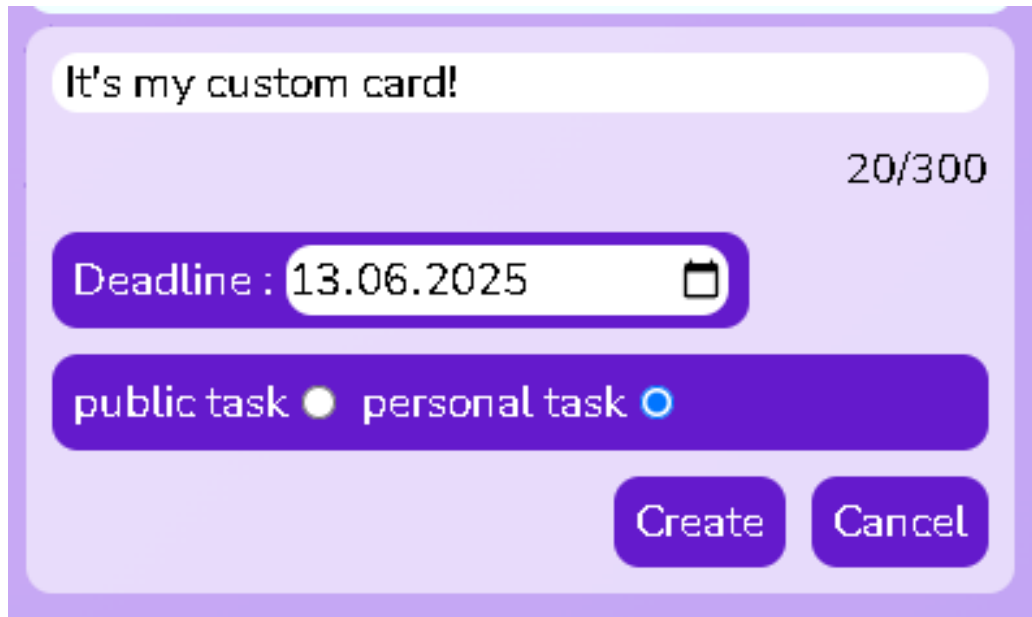


Рис. 5.5: Дошка з завданнями

5.1.4 Інтерфейс створення завдання

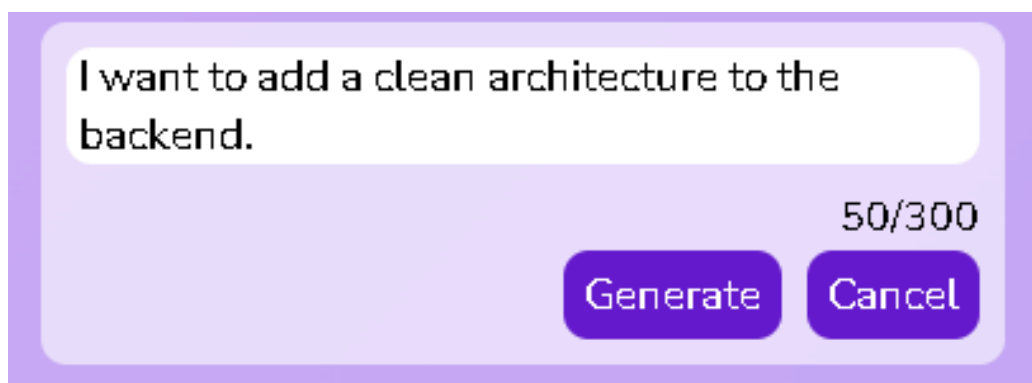
- Вручну:



The screenshot shows a light purple rounded rectangle interface for creating a task manually. At the top, there is a white text input field containing the text "It's my custom card!". To the right of this field, the character count "20/300" is displayed. Below the input field is a dark purple rounded rectangle containing the text "Deadline : 13.06.2025" followed by a calendar icon. At the bottom of this dark purple rectangle are two radio buttons: "public task" with an unselected white circle, and "personal task" with a selected blue circle. At the very bottom of the interface are two dark purple rounded buttons labeled "Create" and "Cancel".

Рис. 5.6: Інтерфейс створення завдання вручну

- За допомогою штучного інтелекту:



The screenshot shows a light purple rounded rectangle interface for creating a task using AI. At the top, there is a white text input field containing the text "I want to add a clean architecture to the backend.". To the right of this field, the character count "50/300" is displayed. At the bottom of the interface are two dark purple rounded buttons labeled "Generate" and "Cancel".

Рис. 5.7: Інтерфейс створення завдання за допомогою штучного інтелекту

РОЗДІЛ 6. Тестування та оцінка системи

6.1 Методологія тестування

Для забезпечення якості розробленого рішення проводилось:

- Модульне тестування окремих компонентів.
- Інтеграційне тестування взаємодії між фронтендом та бекендом.
- Ручне тестування користувацького інтерфейсу.

6.2 Результати тестування

Проведені тести підтвердили:

- Високу продуктивність API.
- Стабільну роботу інтерфейсу.
- Швидкий відгук системи при високих навантаженнях.

6.3 Продуктивність та масштабованість

Завдяки використанню сучасних технологій додаток може обробляти велику кількість запитів із низьким часом відповіді, що дозволяє масштабувати систему під зростаючі потреби користувачів.

РОЗДІЛ 7. Висновки

У даній курсовій роботі було реалізовано систему «Taskly To Do List App», що демонструє принципи побудови масштабованих додатків за Clean Architecture. Робота містить:

- Аналіз предметної області та огляд технологій.
- Детальний опис архітектури системи.
- Розробку та документування REST API.
- Реалізацію користувацького інтерфейсу з демонстрацією функціональних можливостей.
- Проведення тестування та оцінку продуктивності додатку.

Отримані результати свідчать про доцільність використання сучасних технологій та архітектурних підходів, що забезпечують легкість підтримки, масштабованість та високий рівень безпеки розробленої системи. У перспективі планується розширення функціоналу.

Список використаних джерел

- [1] Малюк А.Ю. Основи роботи з LaTeX / А.Ю. Малюк. — К.: Вища школа, 2021. — 156 с.
- [2] ДСТУ 3008:2015. Документація. Звіти у сфері науки та техніки. Структура і правила оформлення.
- [3] Overleaf. PUT-CREEF dissertation template. — [Електронний ресурс]. — <https://www.overleaf.com/latex/templates/put-creef-dissertation-template/syzpgdspfgvr>
- [4] НУВГП. Академічна доброчесність — [Електронний ресурс]. — <https://nuwm.edu.ua/sp/akademichna-dobrochesnistj>
- [5] HASP.NET Core: JWT and Refresh Token with HttpOnly Cookies — [Електронний ресурс]. — <https://alimozdemir.medium.com/asp-net-core-jwt-and-refresh-token-with-httponly-cookies-b1b96c849742>